

プロジェクト概要: AkiNavi HR-AI (アキナビ HR-AI)

1. ビジョン

「案件の空き」と「人材（HR）リソース」をAIで最適に結びつける、高精度・高品質なマッチング基盤。バグゼロを目指した徹底的なテスト、こまめなGit管理、および将来の担当者が即座に再現可能なドキュメント完備をゴールとする。

2. 技術スタック

- **Frontend:** React 19 (Vite 8), TypeScript, Tailwind CSS v4, TanStack Query v5
 - **Backend/DB:** Supabase (PostgreSQL, Edge Functions, Realtime, pg_cron, pg_net)
 - **AI (ブラウザ)** : Google Gemini デフォルト `gemini-2.0-flash` (`VITE_GEMINI_MODEL` で上書き可)
 - **AI (サーバー・自動取り込み)** : Google Gemini `gemini-2.5-flash` — Supabase Edge Function `inbound-email` のみ
 - **AI (切替・フロントのみ)** : `VITE_AI_PROVIDER=gemini / openai` — OpenAI は未実装スタブ
 - **メール自動取り込み (現行・稼働中)** : Microsoft Graph API ポーリング + Supabase pg_cron (Make.com不要・完全無料・5分間隔)
 - **メール自動取り込み (旧・現在停止中)** : Make.com → Pipedream (いずれも無料枠超過により運用停止)
 - **Testing:** Vitest, React Testing Library, MSW (Mock Service Worker)
 - **Deployment:** Vercel (Frontend), Supabase (Backend)
-

3. 開発・品質管理工程表 (人間とClaude Codeの共同作業)

【Phase 0】 リポジトリ準備・初期化・インフラ開通

1. **[人間] 手作業:** GitHubでPrivateリポジトリ作成、Vercelインポート、APIキー取得
2. **[Claude] 作業:** Vite + React + TypeScriptプロジェクト初期化、主要ライブラリインストール、Git初期化・push

【Phase 1】 DB基盤と環境構築

1. **[Claude] 作業:** jsonbとGINインデックスを用いた汎用DB設計(SQL)の提示
2. **[人間] 手作業:** SupabaseでSQL実行、.env / Vercel環境変数の設定
3. **[Claude] 作業:** 接続確認後、DBスキーマ定義をcommit & push

【Phase 2】 コアロジック開発 & 単体テスト

1. **[Claude] 作業:** AI解析エンジン (Edge Functions) およびスコアリングロジックの実装
2. **[Claude] 開発:** 単体テスト (Unit Test) の記述
3. **[Claude] 作業:** テスト通過を確認し、commit & push

【Phase 3】 UI実装 & 結合テスト

1. **[Claude] 作業:** ランキング表示、マッチング画面、提案履歴機能の実装

- 2. [Claude] 開発: 結合テスト (Integration Test) の記述
- 3. [人間] 手作業: 実際のメールデータを用いた解析・マッチング精度の最終検証
- 4. [Claude] 作業: 完了後、commit & push

【Phase 4】 Make.com 連携・UI改善・デモ環境整備 (完了・メール連携は停止中)

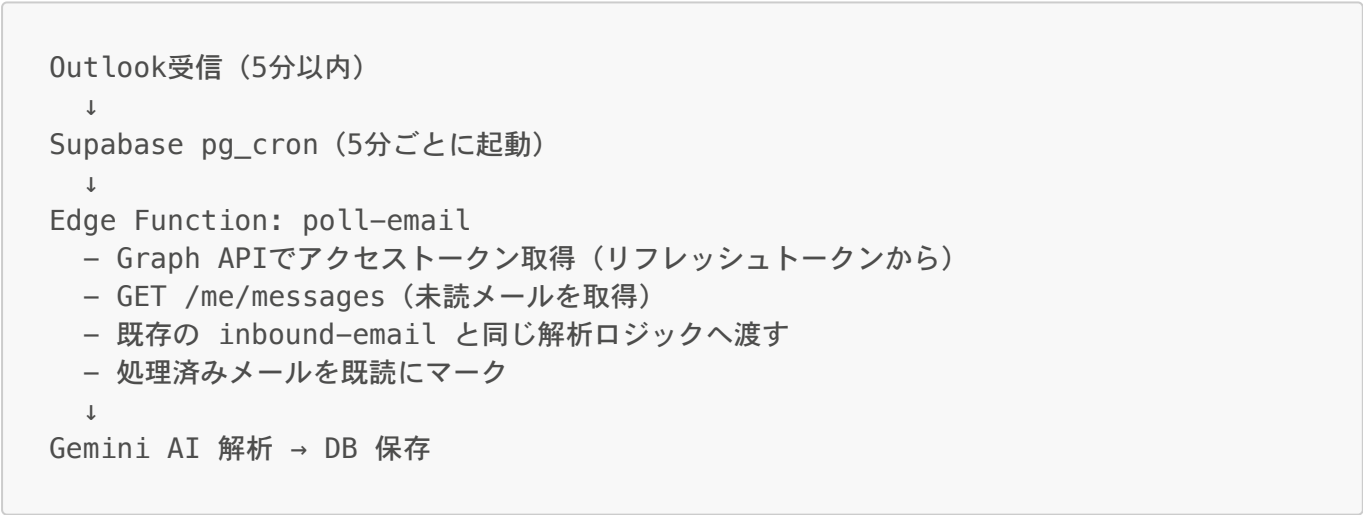
- 1. [Claude] 作業: Make.com (Outlook) と連携した自動解析フローの実装
 - フロー: メール受信 → Make.com が検知 → Edge Function `inbound-email` → Gemini AI 解析 → DB 保存
 - 人材用メール: `akinavi.hr.ai.voice.human@outlook.jp`
 - 案件用メール: `akinavi.hr.ai.voice.project@outlook.jp`
- 2. [Claude] 作業: AI解析精度の継続的改善 (プロンプトチューニング)
- 3. [Claude] 作業: Google Drive / Sheets / Docs リンクの自動取得機能実装
- 4. [Claude] 作業: デモ環境 (data_env) 分離・DemoSeedPanel実装
- 5. [Claude] 作業: 人材・案件の編集機能・最終更新者/日時表示
- 6. [人間] 判断: Make.com・Pipedreamともに無料枠超過 → Microsoft Graph API ポーリングへ移行決定

【Phase 4.5】 Microsoft Graph API ポーリングへ移行 (完了・稼働中)

方針

Make.com・Pipedream等の外部SaaSを廃止し、**完全無料・永続稼働**のメール自動取り込みを実現する。

新フロー:



コスト試算 (すべて無料枠内) :

コンポーネント	無料枠	予想消費量 (5分に1回)
pg_cron	制限なし	約 8,640回/月
pg_net	制限なし	約 8,640回/月
Edge Functions	500,000回/月	約 8,640回/月
Microsoft Graph API	無料	約 8,640回/月

作業一覧

【人間】手作業（Claude Codeでは実施不可）

1. Azureアプリ登録（所要: 約30分）

- <https://portal.azure.com> にアクセス（Microsoftアカウントでログイン・無料）
- 「Microsoft Entra ID」→「アプリの登録」→「新規登録」
- 名前: 任意（例: **akinavi-mail-poller**）
- サポートされるアカウントの種類: 「個人用Microsoftアカウントのみ」を選択
- リダイレクトURL: **<http://localhost>** を追加
- 登録後、以下を控える:
 - **クライアントID（アプリケーションID）**
 - 「証明書とシークレット」→「新しいクライアントシークレット」→ **クライアントシークレット（値）**
- 「APIのアクセス許可」→「アクセス許可の追加」→ Microsoft Graph → 委任されたアクセス許可
 - **Mail.Read** を追加
 - **Mail.ReadWrite**（既読マーク用）を追加

2. リフレッシュトークン取得（2アカウント分）（所要: 約30分）

- 以下のURLをブラウザで開き、**human@outlook.jp** でログイン:

```
https://login.microsoftonline.com/consumers/oauth2/v2.0/authorize
?client_id=<クライアントID>
&response_type=code
&redirect_uri=http://localhost
&scope=offline_access Mail.Read Mail.ReadWrite
```

- ログイン後にリダイレクトされるURL（**<http://localhost/?code=...>**）から **code=** の値を控える
- Claude Codeに渡してリフレッシュトークンに交換（次のClaude作業で実施）
- **project@outlook.jp** でも同様に繰り返す

3. Supabase Secrets に登録（所要: 約10分）

- Supabase Dashboard → Project Settings → Edge Functions → Secrets
- 以下を追加:

シークレット名	値
GRAPH_CLIENT_ID	AzureのクライアントID
GRAPH_CLIENT_SECRET	Azureのクライアントシークレット
GRAPH_REFRESH_TOKEN_HUMAN	human@outlook.jp のリフレッシュトークン
GRAPH_REFRESH_TOKEN_PROJECT	project@outlook.jp のリフレッシュトークン

4. SupabaseでSQL実行（所要: 約5分）

- **[supabase/migrations/add_email_polling_cron.sql](#)** をSQL Editorで実行

- pg_cron・pg_net の有効化とスケジュール登録

【Claude Code】作業（完了）

1. Edge Function poll-email の実装

- supabase/functions/poll-email/index.ts を新規作成
- 4アカウント（human/prod, project/prod, human/demo, project/demo）を Promise.allSettled で並列処理
- リフレッシュトークンを app_config テーブルでローテーション保存（フォールバック: Secrets）
- resolveCallKey(): INBOUND_CALL_KEY → SUPABASE_SERVICE_ROLE_KEY → SUPABASE_ANON_KEY の順で eyJ 始まりJWTを使用
- 処理失敗時は markAsUnread で元に戻し次回再試行

2. pg_cronマイグレーションSQL作成

- supabase/migrations/add_email_polling_cron.sql を作成
- pg_cron・pg_net の有効化、app_config の UNIQUE 制約追加
- 5分ごとに poll-email を起動するスケジュール登録

3. 動作確認: 4アカウント全て正常稼働（未読メール → 解析 → DB保存 → 既読マーク）

【Phase 5】最終納品ドキュメント作成（未着手）

1. [Claude] 作業: システム構成図のメンテナンス（README.md に Mermaid 図あり）
2. [Claude] 作業: 操作マニュアルのメンテナンス（docs/Sales_Manual.md / docs/Sales_Manual.pdf ・営業担当者向け）
3. [Claude] 完了: 全成果物を commit & push し、納品完了

4. Claude Codeへの重要な行動指針

- 正の所在: 仕様・挙動の優先順位は **本リポジトリのソース** と **README.md**。本ファイル（**CLAUDE.md**）はそれに追従するメモであり、食い違いがあれば **ソースを正として本ファイルを更新**すること。
- こまめな Git 操作: 機能実装単位、またはテスト通過ごとに、意味のあるメッセージと共に **commit & push** を行うこと。
- バグゼロの追求: ロジックには必ずテストコードを付随させ、テスト項目書をエビデンスとして出力すること。
- ドキュメントの対象読者:
 - README/構成図は「後任エンジニア」が最短で再現できるように。
 - 操作マニュアルは「非IT営業職」がIT用語なしで理解できるように。

5. データベース構成

candidate_skills のカテゴリ CHECK 制約は **supabase/migrations/add_candidate_skills.sql** を正とする。

テーブル一覧

テーブル	用途
candidates	人材マスタ。スキル・経歴・raw_profile を保持。 data_env (prod demo) で論理分離
projects	案件マスタ。必要スキル・予算・raw_data を保持。 data_env 同上
submissions	マッチング提案履歴。スコア・AI要約を保持。 data_env 同上
candidate_skills	スキルをカテゴリ別に分解して保持（検索最適化・14カテゴリ）
ai_logs	AI解析の実行ログ（モデル・所要時間・結果・エラー）
app_config	アプリ全体設定

candidate_skills の14カテゴリ

カテゴリ	内容
languages	プログラミング言語・クエリ言語
frameworks	フレームワーク
libraries	ライブラリ、UIキット等
os	OS
databases	RDB・NoSQL・KVS
dwh	DWH・BI 等
clouds	クラウドサービス
infrastructures	インフラ技術（コンテナ・IaC 等）
tools	Git, Jira, Slack, Notion, BIツール等
methodologies	PM・マネジメント系
certifications	資格試験等
design	デザイン・クリエイティブ系
marketing	マーケティング・集客系
others	その他

6. 実装済み機能の詳細

メール自動受信（現行・ポーリング方式）

- **Edge Function:** `supabase/functions/poll-email/index.ts`（Phase 4.5 で実装）
- **スケジューラ:** Supabase pg_cron（5分ごとに起動）
- **認証:** Microsoft Graph API（OAuthリフレッシュトークン方式）
- **人材用:** `akinavi.hr.ai.voice.human@outlook.jp`
- **案件用:** `akinavi.hr.ai.voice.project@outlook.jp`

- 処理済みメールは既読マークで重複取得を防止

メール自動受信（旧方式・停止中）

- **Make.com:** 無料枠 約1,000ops/月 → 超過により停止
- **Pipedream:** 無料枠 10クレジット/日 → 当日中に超過・停止
- **Power Automate:** 外部URLへのHTTP POSTがプレミアムコネクタのため不採用

Edge Function `inbound-email` (`supabase/functions/inbound-email/index.ts` 準拠)

- **データ環境:** ボディまたはクエリの `mode / data_env` (`prod | demo | dev`)。省略時は `prod`
- **Secrets:** `GEMINI_API_KEY`、`SUPABASE_URL`、`SUPABASE_SERVICE_ROLE_KEY`
- **`GEMINI_INBOUND_TIMEOUT_MS`:** 1回のGemini待ち上限（未設定時 38,000ms）
- **`INBOUND_MAKE_SOFT_FAIL`:** 例外時も `HTTP 200 + ok:false` で返す（Make/外部サービス停止回避）
- 本文・添付とも空: `HTTP 200 + skipped`

論理データ環境 `data_env`

- `prod / demo` を同一Supabase内で分離（`data_env` カラムでフィルター）
- **デモ解除:** `VITE_DEMO_KEY` と URL クエリ `?demo=<鍵>` でトグル
- 解除時にヘッダの「データ」セレクトが表示される
- デモ UI 未解除時は常に `prod` 固定

マッチング画面

- **実行モード:** `fast`（高速・上限あり） / `full`（全件）
- **高速モード上限:** 案件あたり候補 最大20名、人材あたり案件 最大10件
- **一括マッチングのキャンセル:** `bulkCancelRequestedRef` による途中停止対応
- **`MatchingPage` は常時マウント:** タブ切替で `mutation` が中断されないよう `hidden` で切替

デモシード (`src/components/DemoSeedPanel.tsx`)

- デモ環境向けに人材・案件のサンプルペアをDB投入

画面構成

- タブは3つに整理（`マッチング結果 / 人材登録 / 案件登録`）
- `提案履歴 / 重複管理 / 解析監視` は実装済みだがナビから非表示

Google Drive / Sheets / Docs 自動取得

- メール本文中のリンクを自動検出・取得
- Sheets → CSV、Docs → txt、Drive PDF → base64化してGeminiに渡す
- 認証不要（リンクを知っている全員が閲覧可の共有設定前提）

AI プロバイダー

- **ブラウザ:** Gemini（既定 `gemini-2.0-flash`）

- サーバー (Edge Function) : Gemini `gemini-2.5-flash` 固定
- フロントは `AIProvider` インターフェースで抽象化 (OpenAI切替はスタブのみ)

認証・ニックネーム制

- ログイン機能なし
- 初回アクセス時にニックネームを入力させ `localStorage` に保存

データ重複管理

- email が同じ場合は自動で既存レコードを UPDATE (上書き更新)
- AI が重複疑いと判断した場合は `duplicate_flag = true` を立てるだけ (自動マージ不可)

AI解析ログ (ai_logs)

- 全AI解析呼び出しをDBに記録 (モデル名・所要時間ms・結果JSON・エラー)
- 成功・失敗どちらもログに残す